



Level Integration Connector

A Resilient, Idempotent Workforce Sync System

Monorepo Architecture Overview

Built on a modern, type-safe structure ensuring consistency and reliability between components.



**NestJS
Backend**

Powers core business logic with robust, dependency-injection-friendly framework.



**Next.js
Frontend**

Delivers a reactive, modern dashboard for monitoring and interaction.



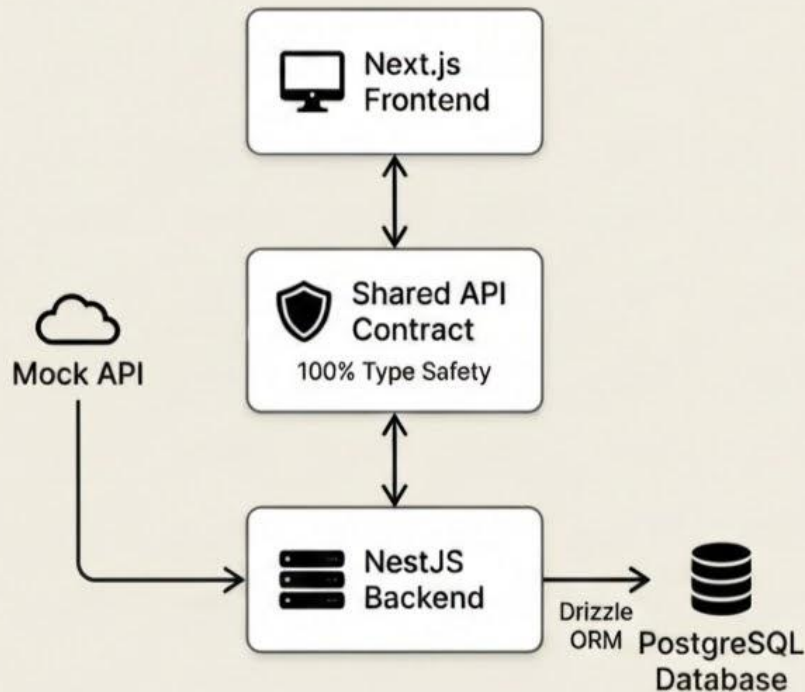
**Shared API
Contract**

Dedicated package guaranteeing 100% type safety between frontend and backend.



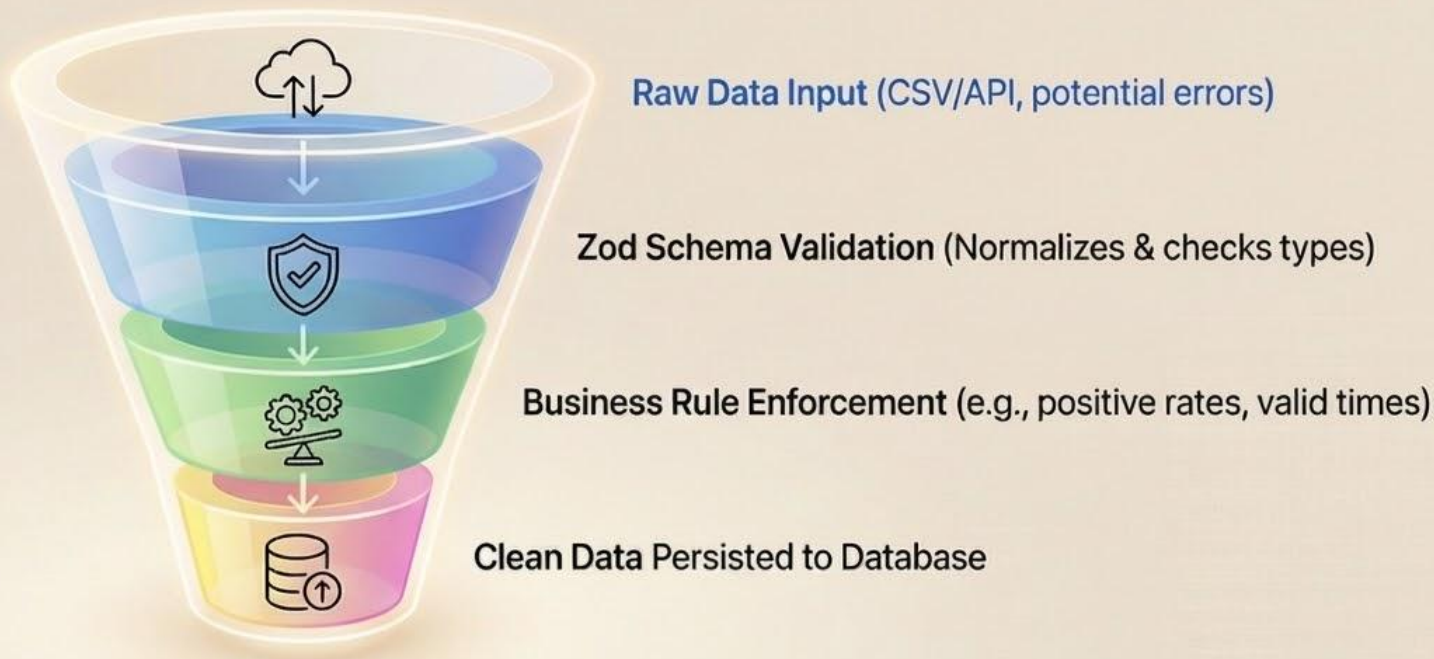
**PostgreSQL &
Drizzle ORM**

Provides idempotent persistence, ensuring data integrity and safe retries.



Data Ingestion Pipeline

Our validation pipeline, powered by Zod, acts as a robust shield, ensuring only clean and valid data reaches the database. It normalizes imperfect data and enforces strict business rules with clear, row-level error reporting.



Derived Fields & Temporal Safety



Accurate Duration Math

Work duration is calculated in minutes using ISO epoch math. Reliably handles midnight crossings, preventing rollover errors.



Temporal Overlap Prevention

Strict database queries prevent overlapping shifts, ensuring data integrity and preventing double payments.



Integer-Based Wage Calculation

To avoid floating-point precision errors, wage calculations are performed using integers.

£12.50 → 1250 cents

Rate converted to and stored as integers.

GRACEFUL FAILURE: Partial Success vs. Total Failure

A Real-World Design Decision: Data Syncing

1.



Error Detection: 1 out of 10,000 records corrupt. Do not crash sync.

2.

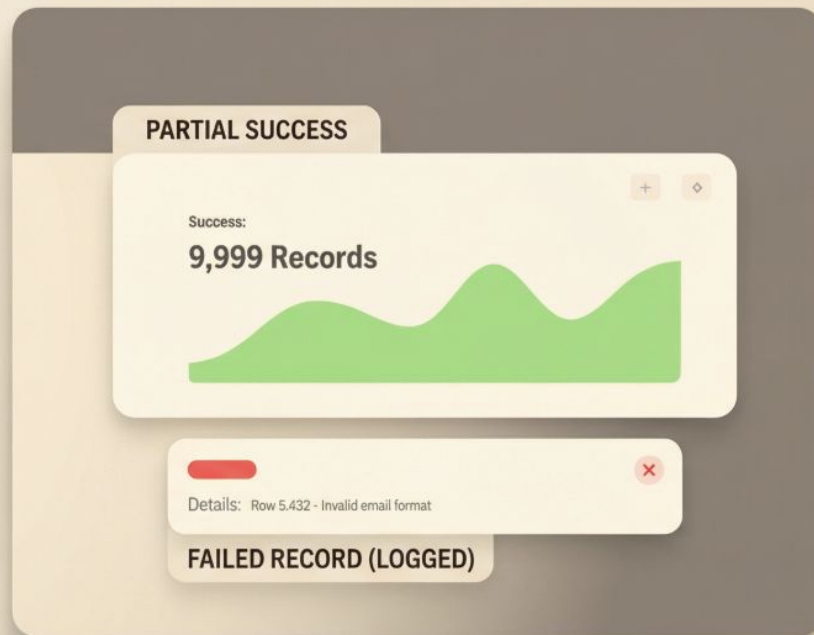


Partial Processing: 9,999 workers continue. Ensure business continuity.

3.



Error Logging: Log exact error & problematic row (e.g., 'SyncRun' record) for review.



Reliability, Telemetry & Developer Experience



Jest Unit Testing Coverage

Core business logic thoroughly covered. Includes overlap logic, mathematical boundaries, and upsert behavior.



Log, Metrics and Traces

Structured error logs and metrics to troubleshoot errors easily.



Comprehensive Documentation

Detailed 'README.md' covers architectural tradeoffs and explicitly handled edge cases.

```
$ jest test runner
> PASS src/business-logic.test.ts

$ drizzle-kit push:pg
> Pushing schema to 'default'...
```

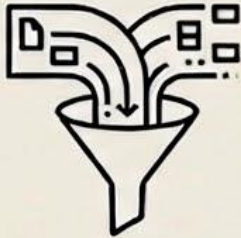
README.md

- ## Architecture
- ## Tradeoffs
- ## Edge Cases

The Road Ahead: Scaling

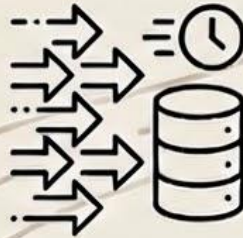
Non-Functional Requirements (Qualities of the System)

**Handle Massive Data
with Ease**



Instead of trying to process huge files all at once, we'll move to a "streaming" model. This ensures the system stays stable and never crashes, even when handling millions of rows.

**High-Speed
Updates**



We will transition from saving records one-by-one to "bulk processing." This makes data synchronization significantly faster and reduces the load on our database.

**Instant
Responsiveness**



By moving heavy data work into a background "Job," the app stays lightning-fast for users. They can trigger a sync and keep working while the system handles the heavy lifting behind the scenes.



Thank You!

Questions & Feedbacks

